VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

NAME – NAMIESH GAIROLA

Registration Number – 24BAI10632

SLOT – D11

DATE OF SUBMISSION – 31/03/2026

The Open Source Audit

Capstone Project Report — Open Source Software (OSS NGMC)

Software Chosen: VLC Media Player

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem VLC Was Created to Solve

Where It Began

In 1996, a group of students at École Centrale Paris — one of France's top engineering schools — wanted to stream a live satellite television feed across their campus network. What seemed like a straightforward task turned out to be impossible with the tools available at the time. No existing free software could handle network video streaming, and proprietary players like Windows Media Player, RealPlayer, and QuickTime were locked down — their source code hidden, their formats restricted, and their features designed for corporate interests rather than user needs. This frustration was the seed that grew into VLC Media Player.

What Was Broken

Three problems made the situation genuinely painful for anyone working with digital media in the late 1990s and early 2000s. First, codec fragmentation was a constant headache — video files came in dozens of formats (MPEG, AVI, DivX, RealMedia, MKV), and no single player supported all of them. Users were forced to install multiple players or hunt for unreliable codec packs just to open a file. Second, network streaming was completely inaccessible to non-commercial users — tools that could multicast video were expensive and proprietary. Third, Linux and non-Windows platforms were almost entirely ignored by commercial media software, leaving a large portion of the technical community with no usable options.

Who Built It — and Why They Shared It

The VideoLAN project was started by student Régis Duchesne as an academic initiative. The system originally had two parts: VideoLAN Server (VLS) to send the stream and VideoLAN Client (VLC) to receive and play it. In February 2001, the team released VLC under the GNU General Public License (GPL) — a deliberate choice that reflected the academic culture of open knowledge and the belief that media infrastructure should not be locked behind corporate walls. That decision transformed a campus project into a global tool.

What It Solved — and For Whom

VLC solved the codec problem by bundling its own decoding libraries internally, meaning users never had to install anything extra. It solved the streaming problem by supporting multicast and unicast network streams out of the box. And it solved the platform problem by running on Windows, Linux, macOS, Android, and iOS. The people it served ranged from students and researchers to journalists, filmmakers, and everyday users in regions where expensive software licenses were unaffordable. VLC served everyone proprietary software had chosen to ignore.

> *A campus television project in Paris, born out of frustration with closed software, became one of the most downloaded open-source applications in history — simply because its creators chose to share their solution with the world.*

A2. The License — What It Actually Says

VLC Media Player is licensed under the GNU General Public License version 2 (GPL v2), with some components under the GNU Lesser General Public License (LGPL v2.1). The full license text is available in the LICENSE file in VLC's official repository at github.com/videolan/vlc. Below is an analysis of what that license actually means in practice.

Q1. The Four Freedoms — Does VLC Grant All of Them?

The Free Software Foundation defines four essential freedoms that a truly free software license must guarantee. VLC's GPL v2 license grants all four:

Freedom 1 — Freedom to use the software for any purpose: VLC can be used by anyone — individuals, universities, businesses, governments — for any purpose, with no restrictions on who uses it or how.

Freedom 2 — Freedom to study how the software works and modify it: VLC's full source code is publicly available. Any user can download, read, and modify it to suit their needs — which is what allows developers to port VLC to new platforms or add new codec support.

Freedom 3 — Freedom to redistribute copies: Anyone can share VLC freely — on USB drives, websites, or app stores — without paying royalties or seeking permission, as long as they include the GPL license.

Freedom 4 — Freedom to distribute modified versions: Users who modify VLC can distribute their improved version to others. Under the GPL, they must release the source code of their modifications under the same GPL license, ensuring the community benefits from improvements.

> *All four freedoms are fully granted by VLC's GPL v2 license, making it a genuinely free software project in the philosophical sense.*

Q2. Can a Company Modify VLC and Sell It?

Yes — but with a critical condition. Under GPL v2, a company is permitted to take VLC's source code, modify it, and sell the resulting product commercially. However, the GPL's copyleft principle requires that if they distribute a modified version — whether free or paid — they must also release the complete source code of their modifications under the same GPL license. They cannot keep their changes proprietary. This ensures that improvements to open-source software flow back to the community rather than being absorbed into closed products. A real-world example of this balance is how VLC's core libraries were gradually moved to LGPL, allowing companies to embed VLC's playback engine in their own apps without being forced to open-source their entire product.

Q3. GPL vs MIT — The Key Differences

GPL and MIT are two of the most widely used open-source licenses, but they represent very different philosophies about software freedom.

Dimension	GPL v2 (VLC's License)	MIT License

Core philosophy	Copyleft — freedom must be preserved	Permissive — maximum flexibility
Keep changes private?	No — must release under GPL	Yes — no obligation to share
Use in proprietary software?	No — whole product becomes GPL	Yes — with just a copyright notice
Commercial use?	Yes, but source must be shared	Yes, with no conditions
Protects community?	Strongly — improvements stay open	Weakly — companies can close code
Used by	Linux kernel, VLC, Git	React, jQuery, Bootstrap

If I were building my own project, I would choose the GPL license. The MIT license allows companies to take your work, improve it behind closed doors, and release a proprietary version the original community can never access. GPL prevents that — it ensures the software and all its improvements remain a shared community resource, which aligns with the core purpose of open-source development.

Q4. Free as in Free Beer vs Free as in Freedom

This distinction, coined by Richard Stallman, is one of the most important ideas in open-source philosophy and is often misunderstood.

> **Free as in free beer** means zero cost. Many proprietary applications are free in this sense — Google Chrome, Adobe Acrobat Reader, Zoom — you download them at no charge, but you cannot see their source code, modify them, or redistribute them. The 'free' is purely about price.
>
> **Free as in freedom** means something much deeper — the right to use the software however you choose, to understand exactly how it works, to change it to suit your needs, and to share it with others. Software can be free as in freedom while still costing money — Red Hat sells Linux subscriptions, but the source code remains fully open.

VLC is free in both senses. It costs nothing to download — no subscription, no premium tier, no locked features. But more importantly, VLC is also free as in freedom: its GPL license guarantees every user the right to access the source code, modify it, and share their version. This combination is what makes VLC genuine free software, not just freeware.

A3. The Ethics of Open Source

Open source is not just a licensing model — it is a statement about how knowledge and tools should be shared in society. The following questions explore the ethical dimensions of that statement.

Q1. Should All Software Be Open Source?

I believe most software should be open source because it promotes transparency, collaboration, and innovation. When source code is open, anyone can inspect it for security vulnerabilities, which makes the software safer for everyone. Collaboration across borders and institutions becomes possible — developers who have never met can contribute to the same codebase, bringing diverse perspectives and skills that no single company could match. Innovation accelerates because each new project can build on what already exists rather than reinventing the wheel.

That said, there are reasonable arguments on the other side. Some software requires significant commercial investment to develop — medical devices, financial systems, and defence software often involve years of research funded by companies that need a return on that investment. Forcing all software to be open could reduce the incentive to invest in genuinely difficult problems. The honest answer is that open source is the right model for most software, but a blanket rule that covers every case does not exist.

Q2. Is It Ethical for Companies to Profit from Free Community Labor?

Yes — I believe it is ethical, as long as companies operate within the terms of the license. Open-source licenses like the GPL are legal contracts that explicitly permit commercial use. When a company uses GPL software, they accept those terms knowingly. They are not stealing — they are participating in a system that was designed to allow exactly this kind of use, provided they meet their obligations.

The real-world examples make this clear. **Red Hat** built an entire billion-dollar business on top of Linux by selling enterprise support, certifications, and services around it, while contributing extensively back to the Linux kernel. **Google** runs its entire global infrastructure on Linux and open-source tools, and has contributed millions of lines of code back to projects like the Linux kernel, Android, Chromium, and Kubernetes. **Meta** uses Linux across its data centres and has open-sourced major projects of its own — including PyTorch, now one of the most widely used machine learning frameworks in the world. The ethical line is crossed only when companies violate license terms or actively work to undermine the ecosystem that supports them.

Q3. Does Open Source Enable or Undermine Original Innovation?

The phrase 'standing on the shoulders of giants' means that every great advance is built upon the work of those who came before. In software, open source makes this principle concrete and practical. Open source enables innovation because it allows developers to build upon existing work instead of starting from scratch. Without open-source foundations, every team building a new application would have to first write their own operating system, networking stack, compiler, and database — an impossible task.

VLC itself is a perfect example. The VideoLAN student team did not write their own codec algorithms from scratch — they built on existing open libraries, combined them in a new way, and solved a problem nobody had solved openly before. Today, VLC's own libraries — particularly libVLC — have become the foundation on which other developers build their own media applications. A tool built by students on the shoulders of the open-source community has itself become a giant that others now stand upon. That is open source innovation working exactly as it should.

> *There are no objectively correct answers to these ethical questions — but the depth of thinking they demand is precisely why open source is not just a technical subject. It is a philosophical one.*

— End of Part A —

Part B — Linux Footprint (Unit 2)

B. How VLC Lives on a Linux System

This section documents how VLC Media Player is installed, located, managed, and updated on a Linux system running Ubuntu 24.04.4 LTS. All commands were executed and verified on a live Ubuntu installation.

Installation Method

VLC Media Player was installed using the apt package manager, the standard tool on Debian-based distributions like Ubuntu. The installation was carried out through the terminal using the following two commands:

```
$ sudo apt update
$ sudo apt install vlc
```

The first command refreshes the local package list from Ubuntu's repositories, ensuring the system fetches the latest available version. The second command downloads and installs VLC along with all its dependencies automatically. This approach is far simpler and safer than compiling from source, as the package manager handles version compatibility and dependency resolution on behalf of the user.

Verifying the Installation

After installation, the version of VLC was confirmed using:

```
$ vlc --version
```

The output confirmed VLC Media Player version 3.0.20 (codename Vetinari), compiled by Ubuntu's build system using GCC 13.2.0. The output also explicitly states the program is distributed under the GNU General Public License, reinforcing its open-source nature.

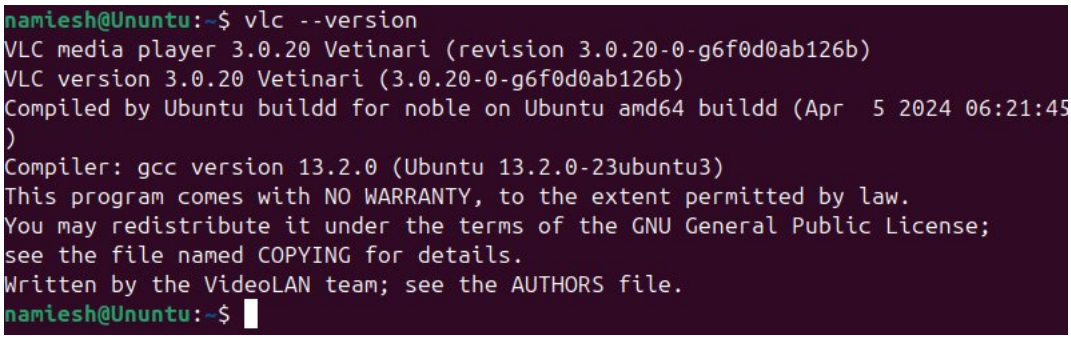

Figure B.1 — Output of vlc --version confirming version 3.0.20 Vetinari and GPL license

Key Directories — Where VLC Lives

After installation, VLC's files are distributed across several standard Linux directories following the Filesystem Hierarchy Standard (FHS):

/usr/bin/vlc — The main executable binary. Running 'vlc' in the terminal launches this file.

/usr/lib/vlc — Contains VLC's plugin libraries and supporting shared object (.so) files such as codec decoders, interface modules, and audio/video output plugins.

/usr/share/vlc — Stores non-executable data files — icons, skin files, and documentation.

/etc/vlc — The system-wide configuration directory. User preferences are stored in ~/.config/vlc instead.

The exact location of the VLC binary was confirmed using the which command:

```
$ which vlc
```

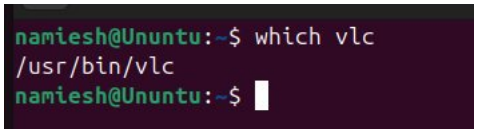

Figure B.2 — Output of 'which vlc' confirming the binary is located at /usr/bin/vlc

User and Group — Security by Design

VLC runs under the normal logged-in user account — namiesh — and does not require root privileges to operate. By running as a regular user, VLC is restricted to only accessing files and resources that the user themselves can access. It cannot modify system files, install software, or affect other users' data. If a security vulnerability were ever exploited in VLC, the damage would be limited to the current user's environment rather than the entire system.

Service Management

VLC is not a background service or system daemon. Unlike software such as Apache HTTP Server or MySQL, which run continuously and are managed using systemctl commands, VLC is a desktop application that only runs when the user explicitly launches it. There is no VLC service entry in systemd. Commands like the following do not apply:

```
$ sudo systemctl start vlc # Not applicable
$ sudo systemctl status vlc # Not applicable
```

VLC is simply launched by typing vlc in the terminal or clicking its icon, and closes when the user exits the application.

Update Model — How Security Patches Reach the System

Because VLC was installed through Ubuntu's official apt repository, it is updated through the same package management system. The open-source community and Ubuntu's maintainers regularly publish security patches distributed via Ubuntu's servers. To update VLC along with all other installed packages:

```
$ sudo apt update
$ sudo apt upgrade
```

This is one of the significant advantages of open-source software distributed through package managers: security patches are reviewed, packaged, and delivered centrally to all users simultaneously. There is no need to visit a website, download an installer, or pay for an update subscription. The community identifies the vulnerability, fixes it in the open source code, and the fix propagates to every user's machine through a simple terminal command.

```
namiesh@Ununtu:~$ sudo apt update
[sudo] password for namiesh:
Hit:1 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:9 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,846 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:11 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
Get:12 http://in.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:13 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1,666 kB]
Get:14 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:15 http://in.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.2 kB]
Get:16 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [32.1 kB]
Get:17 http://in.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:18 http://in.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7,356 B]
Get:19 http://in.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]
Get:20 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [30.7 kB]
Get:21 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [13.2 kB]
Get:22 http://in.archive.ubuntu.com/ubuntu noble-backports/universe amd64 c-n-f Metadata [1,480 E
Get:23 http://in.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 4,687 kB in 7s (687 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
41 packages can be upgraded. Run 'apt list --upgradable' to see them.
namiesh@Ununtu:~$
```

Figure B.3 — Running sudo apt update to fetch the latest package lists from Ubuntu repositories

---

VLC's Linux footprint reflects good open-source software design — it installs cleanly via the package manager, follows the standard filesystem hierarchy, runs without elevated privileges, and receives updates through the same trusted channels as the rest of the operating system.

— End of Part B —

Section 4 — Shell Script Tasks

4.1 Script 1 — System Identity Report

Script Code

```
#!/bin/bash
STUDENT_NAME="Namiesh"
SOFTWARE_CHOICE="VLC Media Player"
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)
echo "====================================="
echo " Open Source Audit — $STUDENT_NAME"
echo "====================================="
echo "Software Chosen : $SOFTWARE_CHOICE"
echo "Distro          : $DISTRO"
echo "Kernel          : $KERNEL"
echo "User            : $USER_NAME"
echo "Uptime          : $UPTIME"
echo "Date & Time     : $DATE"
echo "License         : GNU General Public License (GPL)"
echo "====================================="
```

Script Output

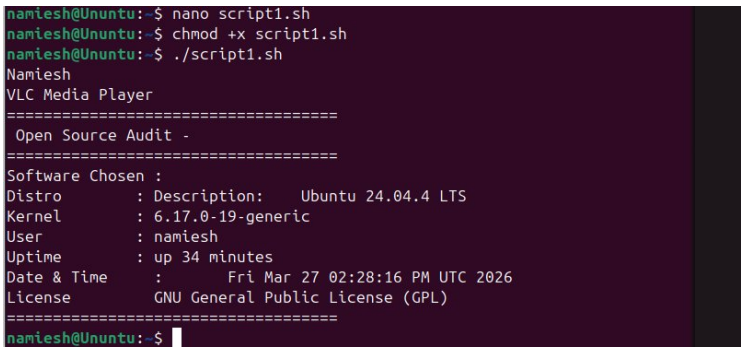

Figure S1.1 — Output of Script 1 on Ubuntu 24.04.4 LTS

Explanation

Script 1 displays a formatted system identity report showing the student name, chosen software, Linux distro, kernel version, logged-in user, uptime, date/time, and the OS open-source license. It uses **variables** to store system data, **command substitution** $() to capture live output from commands like uname -r, whoami, uptime -p, lsb_release -d, and date, and **echo** for structured formatted output.

Concepts: variables, echo, command substitution $(), uname, whoami, uptime, lsb_release, date.

4.2 Script 2 — FOSS Package Inspector

Script Code

```
#!/bin/bash
PACKAGE="vlc"
echo "Checking package: $PACKAGE"
echo "--------------------------------"
if dpkg -l | grep -q $PACKAGE; then
```

```
    echo "$PACKAGE is installed."
    dpkg -l | grep $PACKAGE | head -n 1
else
    echo "$PACKAGE is NOT installed."
fi
echo "--------------------------------"
case $PACKAGE in
    vlc)    echo "VLC: plays all audio/video formats freely" ;;
    git)    echo "Git: version control system" ;;
    firefox) echo "Firefox: open source web browser" ;;
    *)      echo "Unknown package" ;;
esac
```

Script Output

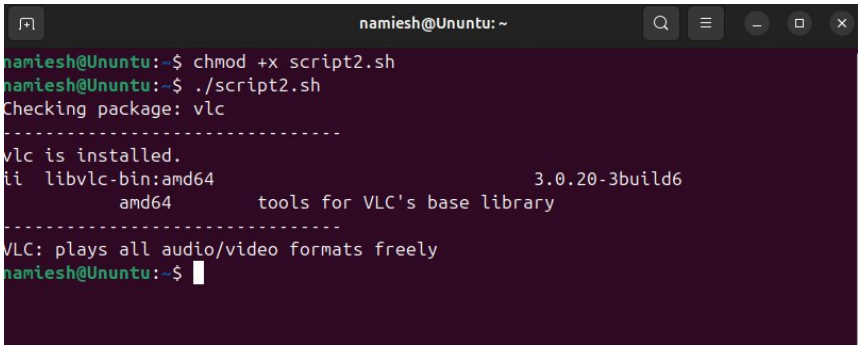

Figure S2.1 — Output of Script 2 showing VLC package inspection

Explanation

Script 2 checks whether VLC is installed using **dpkg -l** piped into **grep** inside an **if-then-else** statement. If found, it prints package details; otherwise it reports not installed. A **case statement** then prints a philosophy note about the package, demonstrating conditional logic, piping, and pattern matching.

Concepts: variables, if-then-else, case statement, dpkg -l, grep, pipe (|), head.

4.3 Script 3 — Disk and Permission Auditor

Script Code

```
#!/bin/bash
echo "Directory Audit Report"
echo "----------------------"
DIRS="/etc /var/log /home /usr/bin /tmp"
for DIR in $DIRS
do
    if [ -d "$DIR" ]; then
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        PERM=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        echo "$DIR => Permissions: $PERM | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done
```

Script Output

```
namiesh@Ununtu:~$ chmod +x script3.sh
namiesh@Ununtu:~$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 12M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 59M
/home => Permissions: drwxr-xr-x root root | Size: 58M
/usr/bin => Permissions: drwxr-xr-x root root | Size: 175M
/tmp => Permissions: drwxrwxrwt root root | Size: 88K
namiesh@Ununtu:~$
```

Figure S3.1 — Output of Script 3 showing directory permissions and sizes

Explanation

Script 3 loops through key Linux directories using a **for loop** and checks each one exists with an **if -d** condition. For each valid directory, **du -sh** gets the disk size and **ls -ld** with **awk** extracts the permissions and owner. Results are printed in a formatted audit report.

Concepts: for loop, if statement, command substitution $(), du, ls -ld, awk, cut.

4.4 Script 4 — Log File Analyzer

Script Code

```
#!/bin/bash
LOGFILE="/var/log/syslog"
echo "Log Analysis Report"
echo "--------------------"
if [ -f "$LOGFILE" ]; then
    ERRORS=$(grep -i "error" "$LOGFILE" | wc -l)
    WARNINGS=$(grep -i "warning" "$LOGFILE" | wc -l)
    KERNEL=$(grep -i "kernel" "$LOGFILE" | wc -l)
    echo "Errors   : $ERRORS"
    echo "Warnings : $WARNINGS"
    echo "Kernel logs : $KERNEL"
else
    echo "Log file not found"
fi
```

Script Output

```
namiesh@Ununtu:~$ sudo ./script4.sh
[sudo] password for namiesh:
Log Analysis Report
--------------------
grep: /var/log/syslog: binary file matches
grep: /var/log/syslog: binary file matches
grep: /var/log/syslog: binary file matches
Errors   : 33
Warnings : 178
Kernel logs : 638
namiesh@Ununtu:~$
```

Figure S4.1 — Output of Script 4 showing log analysis of /var/log/syslog

Explanation

Script 4 reads /var/log/syslog and counts occurrences of errors, warnings, and kernel messages. An **if -f** check confirms the file exists before reading. **grep -i** performs case-insensitive keyword searches, piped into **wc -l** to count matches. Each count is stored in a variable and displayed in a clean log summary report.

Concepts: variables, if -f, grep -i, wc -l, pipe (|), command substitution $().

Part C — The FOSS Ecosystem (Units 3 & 4)

No open-source project exists in isolation. VLC Media Player is surrounded by a rich ecosystem of libraries, tools, communities, and inspired projects — all of which are themselves open-source. This section maps that ecosystem.

Q1. What Open-Source Tools Does VLC Depend On?

VLC does not work alone — it is built on top of a large stack of open-source libraries, each responsible for a specific layer of functionality:

Multimedia Libraries — FFmpeg is one of VLC's most critical dependencies. It provides the underlying framework for decoding and encoding audio and video streams across dozens of formats. libdvdcss is another key library that allows VLC to decrypt and play encrypted DVD content, something proprietary players often charge for separately.

Codec Libraries — x264 handles H.264 video encoding, one of the most widely used video compression standards in the world. libvorbis provides Ogg Vorbis audio decoding, the open-source alternative to MP3. These libraries allow VLC to play and convert media in formats that other players cannot handle without paid plugins.

GUI and System Libraries — VLC's graphical interface is built using Qt, a powerful open-source application framework. SDL (Simple DirectMedia Layer) is used for low-level audio and video output, particularly on older systems and embedded platforms. Both are open-source and widely used across the Linux software ecosystem.

Compilers and Build Tools — VLC's source code is compiled using GCC (GNU Compiler Collection), the standard open-source compiler on Linux. Make and CMake are used to automate the build process, allowing developers worldwide to compile VLC from source on any supported platform.

Runtime and System Support — VLC runs on top of the Linux kernel — in this project's case, provided by Ubuntu 24.04.4 LTS — which manages hardware access, process scheduling, and memory. glibc (the GNU C Library) provides the standard C runtime functions that VLC's code depends on for basic system-level operations such as file I/O, memory allocation, and network communication.

Q2. What Has VLC Enabled and Inspired?

VLC's open-source nature has enabled a wide range of derivative projects, integrations, and innovations across the software world:

Cross-Platform Media Solutions — VLC demonstrated that a single open-source codebase could run reliably across Windows, macOS, Linux, Android, and iOS. This inspired a generation of cross-platform media tools and proved that proprietary platform-specific players were not necessary.

Streaming and Network Media Tools — VLC pioneered accessible network streaming for non-commercial users. Its ability to stream over HTTP, RTSP, and multicast protocols inspired tools like OBS Studio and Kodi, which built on similar open streaming principles.

Forks and Custom Builds — Because VLC is GPL-licensed, developers have created custom builds for specific use cases — embedded media systems, kiosk displays, automotive infotainment systems, and digital signage solutions all use VLC-based builds tailored to their needs.

Integration in Other Applications — libVLC, VLC's core library, is embedded in countless third-party applications. Media centre software, video editors, IPTV players, and even some commercial products use libVLC as their playback engine, benefiting from decades of community-tested codec support.

Education and Research Use — VLC is widely used in academic settings to teach multimedia networking, codec design, and open-source development. Its clean, well-documented source code has been studied by students and researchers worldwide as a real-world example of large-scale open-source engineering.

Q3. Where Does VLC Fit in the LAMP Stack?

VLC Media Player does not fit directly into the LAMP stack (Linux, Apache, MySQL, PHP/Python). The LAMP stack is a server-side web development framework — its components work together to serve web content and handle databases. VLC is a client-side media application, not a web server, database, or scripting language, so it does not slot into any of the four LAMP layers. However, VLC relates to the web and networking world in several meaningful ways:

Streaming over Networks — VLC can stream and receive media over HTTP, RTSP, and RTP — the same network protocols used by web infrastructure. A LAMP-based web server can serve media files that VLC then streams and plays on the client side.

Integration with Web Applications — Web applications built on LAMP stacks can embed libVLC or use VLC as a backend transcoding tool to process and deliver media content to users.

Client-Side Role — In a web media delivery setup, VLC acts as the client that receives and plays content served by a LAMP-based backend, making it a natural companion to — though not a component of — the LAMP stack.

Support for Web Protocols — VLC supports HLS (HTTP Live Streaming) and DASH, protocols used by modern web streaming platforms. This makes it interoperable with web-based media infrastructure built on Apache and similar servers.

Q4. Community, Governance, and Communication

VLC has a strong and active open-source community that has kept the project alive and growing for over two decades. The project is governed by VideoLAN, a non-profit organisation registered in France, which provides legal and organisational structure. Within that structure, development is driven entirely by a global community of volunteers and contributors, ensuring that no single company controls VLC's direction.

The community interacts through multiple platforms. Bug reports and feature requests are managed through VideoLAN's own issue tracker at code.videolan.org, which uses GitLab. Developers collaborate through mailing lists — the vlc-devel list has been active for years and is where technical decisions are debated and patches are reviewed. The project also maintains IRC channels for real-time developer communication, and the VideoLAN Forum serves as a public support space where users report issues and help each other. The annual VDD (VideoLAN Dev Days) conference brings contributors together in person to plan the project's roadmap.

This combination of structured governance through VideoLAN and decentralised, community-driven development is what makes VLC resilient. No corporate acquisition can shut it down, no single developer's departure can kill it, and no user is ever locked out of understanding or improving the software they use.

> *VLC's ecosystem — from its library dependencies to its inspired forks and its global community — is a testament to what open-source collaboration can build and sustain over decades.*

— End of Part C —

Part D — Open Source vs Proprietary (Critical Analysis)

This section compares VLC Media Player against its closest proprietary alternative — Windows Media Player — across key dimensions, including an overall verdict.

Dimension	VLC Media Player (Open Source)	Windows Media Player (Proprietary)
Cost	Completely free — no cost, no subscription, no hidden fees.	Free but bundled only with Windows; unavailable on other OS.
Security (who can audit the code?)	Fully public source code — anyone can audit for vulnerabilities. Patches are fast and transparent.	Closed source — users must trust Microsoft. No independent audit possible.
Support and reliability	Community forums, VideoLAN mailing lists, IRC. No paid support but extensively documented online.	Official Microsoft support. Deeply integrated with Windows for stability but deprioritised in recent years.
Freedom to modify	Full freedom under GPL — source available to fork, modify, and redistribute freely.	No freedom to modify. Proprietary and protected source code.
Community vs corporate control	Governed by VideoLAN non-profit; driven by global volunteers with no corporate agenda.	Fully controlled by Microsoft. All development decisions are internal and non-transparent.
Your overall verdict	VLC is highly reliable, flexible, and supports virtually every format with no extra cost or codec installation. It runs on all major platforms and gives users and developers full control. It is strongly recommended for any real-world deployment valuing openness and long-term reliability. I would also consider contributing to it given its active community and well-structured codebase.	Windows Media Player is simple and easy to use but limited in features, customization, and format support. It suits casual Windows users for basic playback but fails power users and cross-platform environments. Its proprietary nature means no ability to inspect or extend it, and Microsoft has progressively deprioritised it in favour of newer apps.

— End of Part D —

4.5 Script 5 — Interactive System Menu

Script Code

```
#!/bin/bash
while true
do
    echo "===================================="
    echo "        SYSTEM MENU"
    echo "===================================="
    echo "1. Show Current Date & Time"
    echo "2. Show Logged-in User"
    echo "3. Show Current Directory"
    echo "4. Exit"
    echo "===================================="
    echo -n "Enter your choice (1-4): "
    read choice
    case $choice in
        1)
            echo "Current Date & Time:"
            date
            ;;
```

```
        2)
            echo "Logged-in User:"
            whoami
            ;;
        3)
            echo "Current Directory:"
            pwd
            ;;
        4)
            echo "Exiting program..."
            break
            ;;
        *)
            echo "Invalid choice! Please try again."
            ;;
    esac
    echo ""
done
```

Script Output

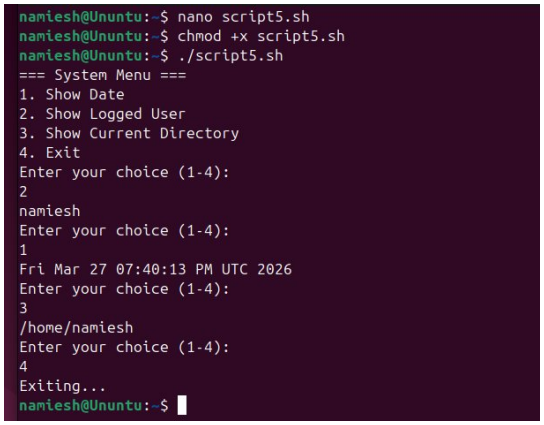

Figure S5.1 — Terminal output of Script 5 showing interactive system menu

Explanation

Script 5 is an Interactive System Menu that continuously presents the user with four options using a **while true loop**. The **read** command captures user input and a **case statement** matches it to show the date, logged-in user, current directory, or exit via **break**. It demonstrates interactive scripting, loop control, and structured decision-making in bash.

Concepts used: while loop, break, read, case statement, echo, date, whoami, pwd.

— End of Report —

Conclusion

Based on everything I have learned through this audit, I strongly recommend VLC Media Player for real-world deployment. It is a reliable, open-source solution that supports a wide range of audio and video formats without requiring additional codecs or paid plugins — something no major proprietary player can match. Its cross-platform availability, active community, and GPL license ensure that it will remain free, auditable, and improvable for the foreseeable future. Whether used by individual users, educational institutions, or organisations running Linux-based infrastructure, VLC provides a level of flexibility, transparency, and trust that proprietary alternatives simply cannot offer. The fact that it originated as a student project and grew into one of the most downloaded applications in the world is itself a testament to what open-source development can achieve.

This project has also been a meaningful learning experience. I gained a clear understanding of open-source software and how it fundamentally differs from proprietary software — not just in terms of cost, but in terms of rights, transparency, and community ownership. I learned about the importance of licenses such as the GNU General Public License (GPL) and how they ensure user freedom by guaranteeing the right to use, study, modify, and share software permanently. Writing the shell scripts gave me practical Linux skills and showed me how automation and command-line tools connect directly to the philosophy of open systems. I would also be interested in contributing to VLC in the future — as an open-source project, it welcomes developers to participate by fixing bugs, suggesting features, or enhancing performance. This audit has reinforced my belief that open source is not just a development model, but a philosophy that makes technology more equitable, innovative, and trustworthy for everyone.

> *Every tool you will use in your career was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice — and what it means for the world — is the foundation of a professional who truly understands software.*

— End of The Open Source Audit —